



HARDWARE ERP · LOCAL DEVELOPER GUIDE

How to Start the App on Your Own Computer

A complete, no-jargon, step-by-step walkthrough — written for someone with no coding background. Every command and every screen shown here was actually run and captured on this machine, not written from memory.

Prepared for: Ram

Project folder: E:\Project\hardware-erp

Machine: Windows 11, Git Bash + PowerShell, Docker Desktop

① Database

PostgreSQL, runs inside Docker

② Backend

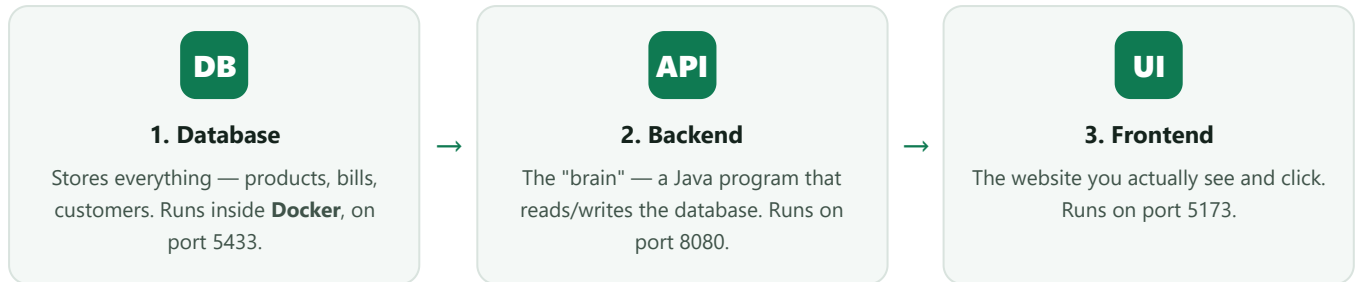
Java server, runs directly on your PC

③ Frontend

The website, runs directly on your PC

The Big Picture

Before touching a single command, it helps to know what you are actually starting. "The app" is not one thing — it is three separate programs that talk to each other over your own computer's network. You will open three things, in this order.



ORDER MATTERS

Always start them in this order: Database → Backend → Frontend. The backend needs the database to already be awake, and the frontend needs the backend. Starting them out of order is the single most common cause of "it's not working" — not a real bug, just impatience.

A few words, explained simply

Terminal

A text window where you type commands instead of clicking icons. You'll use two of them:

Git Bash

and/or

PowerShell

, both already on this PC.

Port

A numbered "door" a program listens on. Two programs can't use the same door on the same computer at the same time — that's the "port already in use" error you've seen.

localhost

A special address that always means "this same computer."

`http://localhost:5173` means "port 5173, on this PC."

Docker

A tool that runs a program (here, the database) inside a small sealed box, so you never have to install PostgreSQL yourself or worry about it conflicting with anything else on your PC.

Process

A running program. Each terminal window you leave open is keeping one process alive. Closing the window kills it.

Log

The stream of text a program prints while it runs, telling you what it's doing — and, if something's wrong, why.

What Needs to Be Installed

Good news — this is a one-time check, and on this machine every one of these was confirmed present and working while writing this guide.

Tool	What it's for	Confirmed on this PC	
Java 21 (JDK)	Runs the backend	21+35-LTS	✓ Installed
Apache Maven	Builds & runs the backend	3.9.16	✓ Installed
Node.js	Runs the frontend	v22.19.0	✓ Installed
Docker Desktop	Runs the database	29.7.2	✓ Installed
Git Bash	Needed for one startup script	Git 2.45.0	✓ Installed

YOU DON'T NEED TO REINSTALL ANYTHING

This checklist is here so that if you ever set this up on a *different* PC, you know exactly what to install first. On this machine, skip straight to Part 3.

The one file that holds your settings — `.env`

The project keeps passwords and settings in a file called `.env`, sitting in the main project folder (`E:\Project\hardware-erp\.env`). It already exists on this PC and is already filled in — you do not need to touch it. It is deliberately excluded from Git (the version-control system) so it never gets uploaded anywhere by accident.

IF YOU EVER SET THIS UP FRESH

Copy the template and fill it in yourself — never delete the real `.env` once it has real values in it:

```
Git Bash
$ cp .env.example .env
# then open .env in Notepad and fill in the blank values
```

Step 1 — Start the Database (Docker)

The database lives inside Docker Desktop. Docker Desktop itself must be running before anything else will work — think of it as the power switch for the whole database box.

1 Open Docker Desktop

Click the Docker whale icon on your Desktop or in the Start menu. Look at the bottom-left corner of the Docker Desktop window, or the small whale icon in the Windows system tray (bottom-right of the screen, near the clock) — once it stops animating and shows a steady icon, Docker is ready. This usually takes 20–60 seconds after opening it.

HOW TO TELL IT'S READY WITHOUT GUESSING

Open a terminal and run `docker info`. If it prints a wall of information about your system, Docker is ready. If it prints a connection error (shown on the Troubleshooting page), it is not ready yet — wait and try again.

2 Open a terminal in the project folder

In File Explorer, go to `E:\Project\hardware-erp`, right-click inside the empty space of the folder, and choose **"Git Bash Here"** (or "Open in Terminal" for PowerShell). Either works for this step.

3 Run the start command

Type this and press Enter. This starts *only* the database — nothing else yet.

```
Terminal — real output, captured on this PC

$ docker compose up -d
Container hardware-erp-postgres Starting
Container hardware-erp-postgres Started
```

The `-d` means "detached" — it starts in the background and gives you your terminal back immediately, instead of filling the screen with database logs forever.

4 Confirm it's actually healthy, not just "started"

"Started" and "ready to accept connections" are not the same moment — Postgres takes a couple of seconds to warm up. Check with:

```
Terminal — real output, captured on this PC

$ docker compose ps
NAME                IMAGE                SERVICE  STATUS
hardware-erp-postgres postgres:16-alpine    postgres Up 7 seconds (healthy)
0.0.0.0:5433->5432/tcp
```

The word **(healthy)** is what you're looking for. If it says `(health: starting)`, wait five more seconds and run the command again.

Step 2 — Start the Backend

The backend is the Java "brain" of the app. It must be started from a Git Bash terminal (not PowerShell) because it uses a small helper script.

DO NOT RUN "MVN SPRING-BOOT:RUN" DIRECTLY

It looks like the obvious command, and it will appear to work — but it silently skips loading the settings from your `.env` file. On this exact project that once caused password-reset emails and login codes to silently vanish for weeks, because nobody noticed the one line in the log saying "Mail not configured." Always use the script below instead — it loads `.env` first, then starts the same backend.

1 Open a new Git Bash window in the project folder

Keep the previous terminal open too (it's not doing anything for the database anymore, but there's no harm leaving it). Right-click in `E:\Project\hardware-erp` → "Git Bash Here".

2 Run the start script

Git Bash — real output, captured on this PC

```
$ ./scripts/run-local.sh
Starting Hardware ERP (local)
  profile   : local
  env file  : /e/Project/hardware-erp/.env
  mail      : Resend selected but RESEND_API_KEY / RESEND_FROM_EMAIL blank - codes and
              links will be LOGGED, not sent
  frontend  : run the Vite dev server in frontend/ in a second terminal

[INFO] Scanning for projects...
[INFO] Building hardware-erp 1.0.0
    ...this part takes the longest – compiling and connecting to the database...
22:33:34 Successfully validated 61 migrations
22:33:34 Migrating schema "public" to version "56 - low stock snapshot"
22:34:48 Tomcat started on port 8080 (http) with context path '/api'
22:34:48 Started HardwareErpApplication in 92.3 seconds
```

THE FIRST RUN IS ALWAYS SLOW — THIS IS NORMAL, NOT A HANG

On this PC the very first start took **92 seconds**. Every start after that, once files are already compiled, took about **44 seconds**. If your cursor just sits there for over a minute the first time, that is expected — do not close the window or press Ctrl+C.

Confirm the backend is actually alive

Leave that terminal window open (closing it stops the backend) and, in any *other* terminal, run:

```
Terminal — real output, captured on this PC

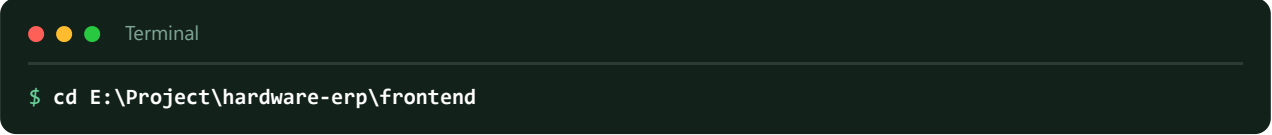
$ curl http://localhost:8080/api/actuator/health
{"status":"UP","components":{"db":{"status":"UP", ...}}}
```

`"status":"UP"` means the backend is running and it can talk to the database. If you don't have `curl`, you can instead just open `http://localhost:8080/api/actuator/health` in your browser and look for the same text.

Step 3 — Start the Frontend

This is the actual website — the part you will click around in. It also needs its own terminal window, kept open the whole time you're using the app.

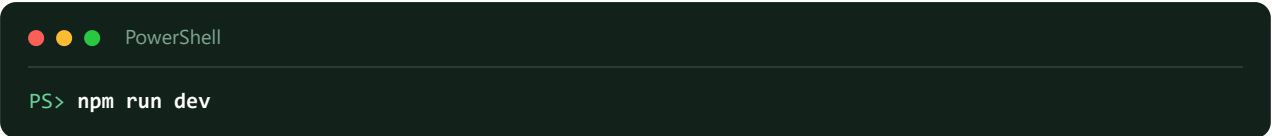
1 Open a third terminal, and go into the frontend folder



```
Terminal  
$ cd E:\Project\hardware-erp\frontend
```

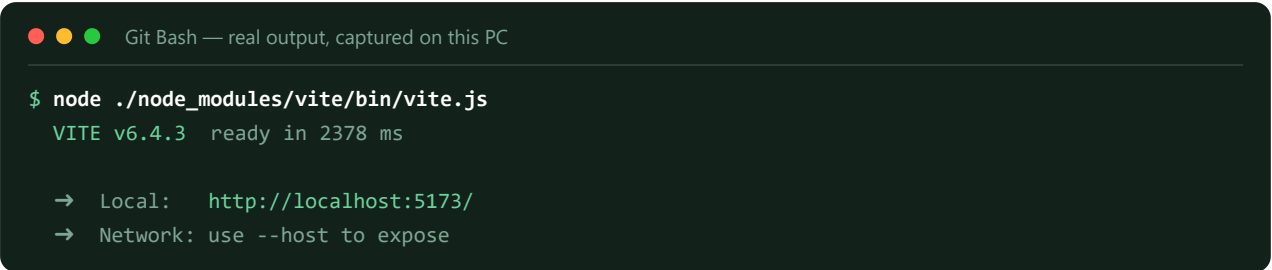
2 Start it

If you're in **PowerShell**, the normal command works fine:



```
PowerShell  
PS> npm run dev
```

If you're in **Git Bash**, that same shortcut sometimes fails with `"node" is not recognized` — a known quirk of this terminal on this PC. Use this instead, which does exactly the same thing:

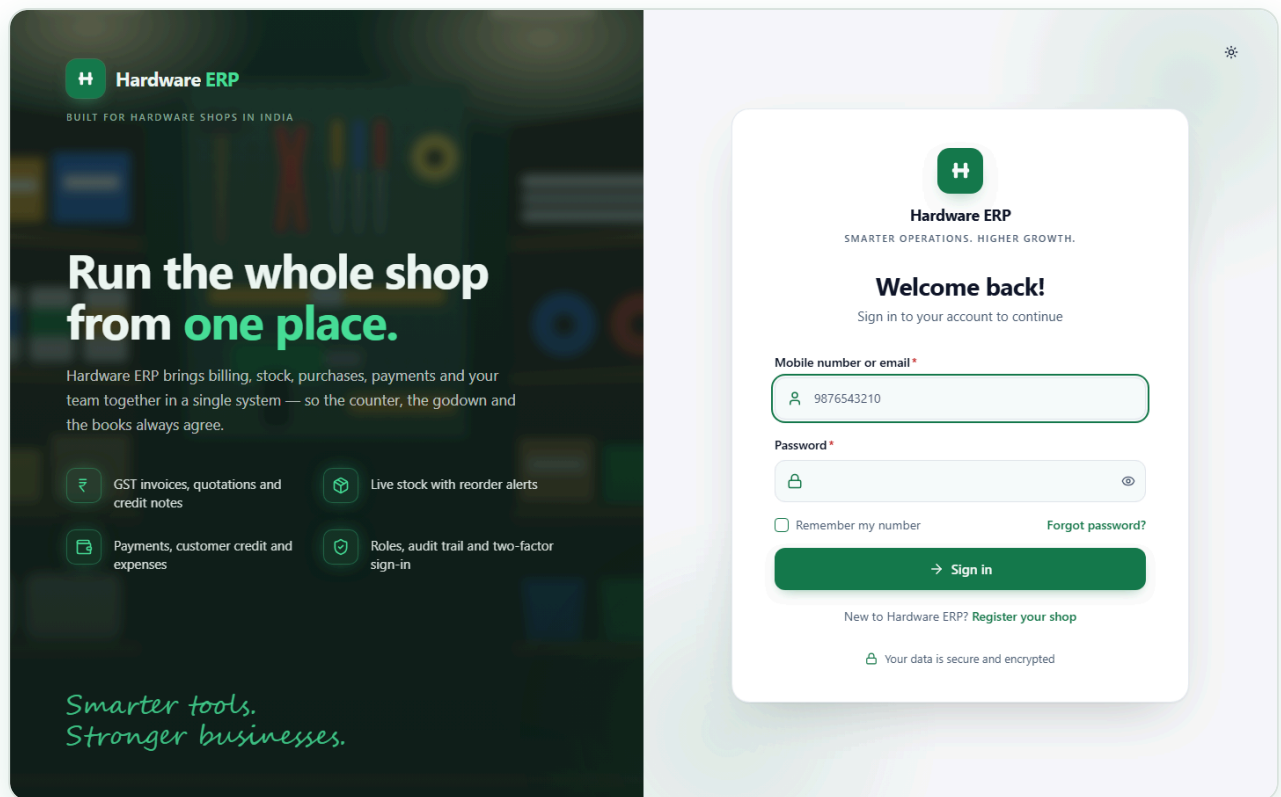


```
Git Bash — real output, captured on this PC  
$ node ./node_modules/vite/bin/vite.js  
VITE v6.4.3 ready in 2378 ms  
  
→ Local: http://localhost:5173/  
→ Network: use --host to expose
```

3

Open the app

Open any browser and go to `http://localhost:5173`. This is the real sign-in screen of this project, captured directly from this running app — not a mockup.



The real Hardware ERP sign-in page, running on this PC during the writing of this guide.

ALL THREE RUNNING AT ONCE = SUCCESS

You now have three terminal windows open: one for the database (or none, since Docker runs it in the background), one running the backend, one running the frontend. Minimize all three and just use the browser — don't close them.

Stopping the App

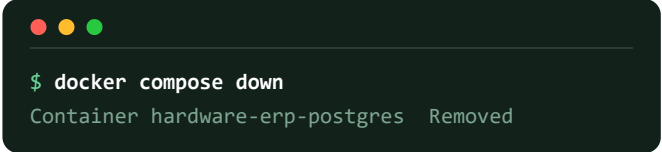
Shut things down in reverse order — it's not strictly required, but it avoids confusing half-finished requests.

Frontend & Backend

Click into each terminal window and press `Ctrl` + `C`. That's it — the process stops immediately. Closing the terminal window entirely does the same thing.

Database

In any terminal, from the project folder:



```
$ docker compose down
Container hardware-erp-postgres  Removed
```

ONE COMMAND TO NEVER RUN BY HABIT

`docker compose down -v` looks almost identical to the safe version above, but the `-v` deletes the actual database contents — every product, bill and customer you've entered — permanently. Only use it on purpose, when you genuinely want to wipe local data and start clean.

YOU DON'T HAVE TO STOP THE DATABASE EVERY TIME

It's fine to leave `docker compose up -d` running for days — it uses very little on an idle machine. Most days you'll only need to redo Steps 2 and 3 (backend, frontend); the database can just stay up.

Docker — What It Is, and When You'd Touch It

You asked specifically how to "connect Docker" — here's the full picture of where it fits, in plain terms.

What Docker actually does here

Docker runs a program inside a small, sealed, disposable "container" — like a shipping container for software. In this project, Docker's *only* job during normal day-to-day development is to hold the PostgreSQL database, so that:

- You never have to install PostgreSQL yourself, configure it, or worry about the version matching.
- It can't quietly break because of something unrelated installed on your Windows PC.
- Deleting it and starting fresh is one command, with zero risk to anything else on your computer.

The backend and frontend are **not** in Docker during normal development — they run directly with Java and Node, because that gives you live reload (the app updates the instant you or Claude change a file) and is much faster to restart than rebuilding a container each time.

The one file that describes it — `docker-compose.yml`

This file (in the project root) is a recipe telling Docker "run PostgreSQL 16, call it `hardware-erp-postgres`, give it port 5433, and remember its data even if the container restarts." You never need to write or edit this file to develop the app — `docker compose up -d` just reads it.

Checking Docker is "connected"

Check	Command	What a good result looks like
Is Docker Desktop running at all?	<code>docker info</code>	Prints system details, not an error
Is our database container up?	<code>docker compose ps</code>	Shows <code>hardware-erp-postgres</code> as <code>Up ... (healthy)</code>
Can the backend reach it?	<code>curl localhost:8080/api/actuator/health</code>	<code>"db":{"status":"UP"}</code>

The advanced case: putting the *whole app* in Docker

There is a second, separate setup — used for actually handing the finished app to a real hardware shop's PC, not for day-to-day development on yours — where Docker runs *all three pieces* together: database, backend and frontend. You won't need this for normal work, but it exists if you ever want to try it:

```
Git Bash — the "everything in Docker" version

$ cp .env.selfhosted.example .env
  # open .env and fill in the 5 required values
$ docker compose -f docker-compose.selfhosted.yml up -d
  # starts the database, backend AND frontend together
  # open http://localhost — nothing after it
```

DON'T MIX THE TWO SETUPS

Never run both `docker-compose.yml` and `docker-compose.selfhosted.yml` at the same time — they were deliberately built as separate Docker projects specifically so one's "wipe and start over" command can never delete the other's data, but running both means two backends and two frontends competing for the same ports 8080/5173/80.

Troubleshooting — Real Errors, Real Fixes

Every error message below is a genuine one, reproduced on this PC while preparing this guide — not a guess at what "might" show up.

"Port already in use" — the backend (port 8080)

This is almost always because an *earlier* backend window is still running somewhere — often minimized and forgotten.

```
Real error, captured on this PC

*****
APPLICATION FAILED TO START
*****

Description:

Web server failed to start. Port 8080 was already in use.

Action:

Identify and stop the process that's listening on port 8080 or configure
this application to listen on another port.
```

Fix, easiest first: look through your open windows for another Git Bash/terminal already running `run-local.sh`, and close it. If you truly can't find it, force it from a fresh terminal:

```
PowerShell — find and stop whatever owns port 8080

PS> netstat -ano | findstr :8080
TCP    0.0.0.0:8080    0.0.0.0:0      LISTENING    18432  ← the last number is the Process ID (PID)
PS> taskkill /PID 18432 /F
SUCCESS: The process with PID 18432 has been terminated.
```

Then run `./scripts/run-local.sh` again.

"Port already in use" — the frontend (port 5173)

Unlike the backend, Vite (the frontend tool) does not treat this as an error — it just quietly moves to the next port:

```
Real output, captured on this PC

Port 5173 is in use, trying another one...

VITE v6.4.3 ready in 1090 ms

→ Local: http://localhost:5174/
```

THIS IS NOT A BUG

Just read the new URL in that message — it will say `5174` , `5175` , or similar. Open *that* address in your browser instead of `5173` .

Troubleshooting — Continued

"Docker is not running" / Docker commands fail

Real error, captured on this PC

```
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine:
check if the path is correct and if the daemon is running
```

Fix: Docker Desktop is simply not open (it does not start automatically with Windows unless you've turned that setting on). Open Docker Desktop from the Start menu, wait about 30–60 seconds for the whale icon to settle, then retry `docker compose up -d`.

Backend script says ".env not found"

The script's own real message

```
ERROR: /e/Project/hardware-erp/.env not found.
Create it from the template: cp .env.example .env
```

Fix: you're most likely in the wrong folder, or the `.env` file got moved/renamed. Run `cd E:\Project\hardware-erp` first, confirm the file is there with `ls -la .env`, and if it's genuinely missing, follow the message — but never overwrite an existing working `.env` with the template.

The website loads, but nothing works (spinning, or network errors)

The frontend (Step 3) can start perfectly well even if the backend (Step 2) isn't running or hasn't finished booting yet — they're independent programs. If the sign-in page appears but clicking "Sign in" does nothing or shows a network error, go back to the backend's terminal window and check for the green `Started HardwareErpApplication` line. If it's not there yet, wait; if the window shows a red error block, read Troubleshooting for that specific error.

Port 5433 (database) already in use / a stale container fights back

This usually means the container from a previous session never fully stopped. Ask Docker what's actually running, and clean up the one occupying the name:

```
$ docker compose down
$ docker compose up -d
```

This restarts cleanly without touching your actual data (only `-v` deletes data — see Part 4).

The backend takes over a minute and you're not sure if it's stuck

It probably isn't. The very first start after a restart measured 92 seconds on this PC (compiling + database migrations + security setup). As long as new lines keep appearing in the terminal every few seconds, it's working. Only worry if the terminal goes completely silent for several minutes *and* shows no red error text.

Quick Reference — Once You've Done It Once

Everything above in three copy-paste blocks. Run each in the folder named, in order, each in its own terminal window that you leave open.

1 Database — any terminal, project root

```
$ cd E:\Project\hardware-erp && docker compose up -d
```

2 Backend — Git Bash only, project root

```
$ ./scripts/run-local.sh
```

3 Frontend — Git Bash, frontend folder

```
$ cd frontend && node ./node_modules/vite/bin/vite.js
```

(PowerShell users: `npm run dev` instead)

→ Then open

`http://localhost:5173`

✕ To stop everything

Ctrl+C in the backend and frontend windows, then:

```
$ docker compose down
```

NEVER, UNLESS YOU MEAN IT

`docker compose down -v` — the `-v` permanently deletes all database data.